

Dynamic

Video-134

Programming



Note :- This playlist is only for explanation of Dns & solutions.

See my "DP Concepts & Dns"
Playlist for understanding
DP from scratch...



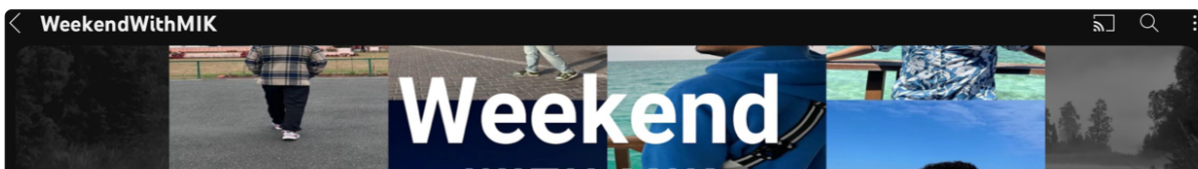
codestorywithmik



CSwithMIK



codestorywithMIK

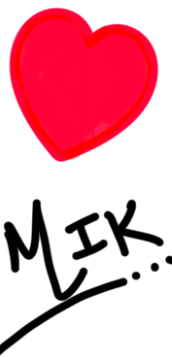




Motivation :-

The best time to plant a tree was twenty years ago,
the second best time is now.

Stop waiting for the perfect conditions and start
building with what you have



3130. Find All Possible Stable Binary Arrays

II

Hard

Topics

Companies

Hint

You are given 3 positive integers `zero`, `one`, and `limit`.

A **binary array** `arr` is called **stable** if:

Same

- The number of occurrences of 0 in `arr` is **exactly** `zero`.
- The number of occurrences of 1 in `arr` is **exactly** `one`.
- Each **subarray** of `arr` with a size greater than `limit` must contain **both** 0 and 1.

Return the *total* number of **stable** binary arrays.

Since the answer may be very large, return it **modulo** $10^9 + 7$.

Example 1:

Input: zero = 1, one = 1, limit = 2

Output: 2

Explanation:

The two possible stable binary arrays are `[1,0]` and `[0,1]`.

Example 2:

Input: zero = 1, one = 2, limit = 1

Output: 1

Explanation:

The only possible stable binary array is `[1,0,1]`.

Example 3:

Input: zero = 3, one = 3, limit = 2

Output: 14

Explanation:

All the possible stable binary arrays are `[0,0,1,0,1,1]`, `[0,0,1,1,0,1]`, `[0,1,0,0,1,1]`, `[0,1,0,1,0,1]`, `[0,1,0,1,1,0]`, `[0,1,1,0,0,1]`, `[0,1,1,0,1,0]`, `[1,0,0,1,0,1]`, `[1,0,0,1,1,0]`, `[1,0,1,0,0,1]`, `[1,0,1,0,1,0]`, `[1,0,1,1,0,0]`, `[1,1,0,0,1,0]`, and `[1,1,0,1,0,0]`.

Constraints :-

$1 \leq \text{zero}, \text{one}, \text{limit} \leq 1000$

Thought Process

FIND ALL POSSIBLE STABLE BINARY ARRAYS I

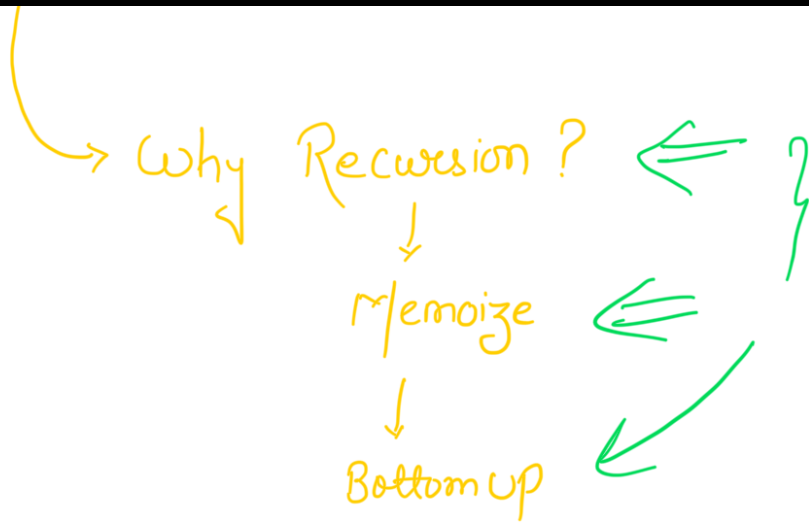
LEETCODE 3129

DETAILED APPROACHES (INTUITION)

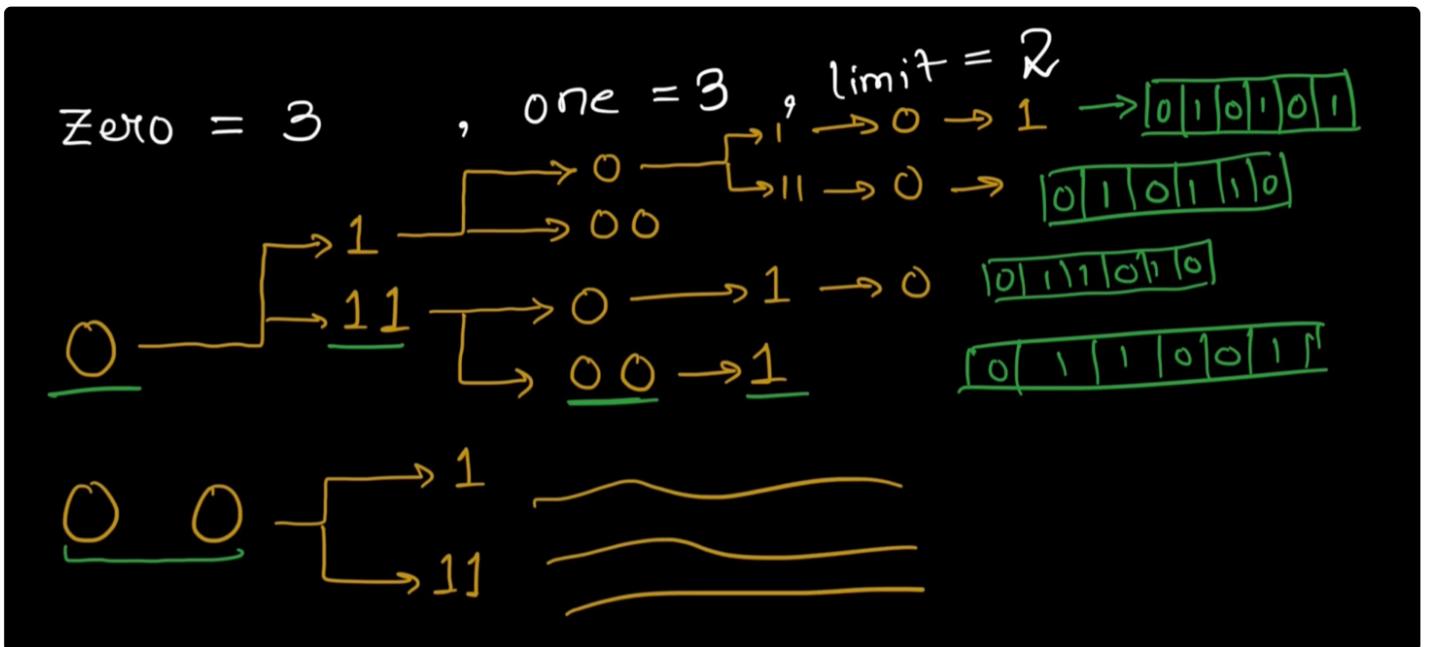
46:58

Find All Possible Stable Binary Arrays I | 2 Approaches | Detailed Intuition | Leetcode 3129 | MIK

codestorywithMIK



Revisiting Part - I



Red Flag :- T.C = $O(\text{Zero} * \text{one} * \text{limit})$

Part-1 : $\underbrace{200} \times \underbrace{200} \times \underbrace{200} = 8 \text{ Million}$

Part-2 : $\underbrace{1000} \times \underbrace{1000} \times \underbrace{1000} = 10^9 \text{ (1 billion)}$

```
for(int onesLeft = 0; onesLeft <= one; onesLeft++) {  
    for(int zerosLeft = 0; zerosLeft <= zero; zerosLeft++) {  
        if(onesLeft == 0 && zerosLeft == 0) continue;  
  
        int result = 0;  
  
        // if(lastWasOne == true) { explore 0s }  
        result = 0;  
        for(int len = 1; len <= min(zerosLeft, limit); len++) {  
            result = (result + t[onesLeft][zerosLeft - len][0]) % M;  
        }  
        t[onesLeft][zerosLeft][1] = result;  
  
        // else { explore 1s }  
        result = 0;  
        for(int len = 1; len <= min(onesLeft, limit); len++) {  
            result = (result + t[onesLeft - len][zerosLeft][1]) % M;  
        }  
        t[onesLeft][zerosLeft][0] = result;  
    }  
}  
  
// startWithOne + startWithZero  
return (t[one][zero][1] + t[one][zero][0]) % M;  
}
```

$O(\text{limit})$

$O(\text{limit})$

$\text{for}(i=0; i \leq \text{one}; i++) \{$
 $\text{for}(j=0; j \leq \text{zero}; j++) \{$

$\text{result} = 0$

$\text{for}(\text{len} = 1; \text{len} \leq \text{min}(\text{onesLeft}, \text{limit}); \text{len}++) \{$

$\text{result} = (\text{result} + t[i][j - \text{len}][0]) \% M;$

$\}$

$t[i][j-1][0]$

$$f[i][j][1] = \text{result}; \quad \left. \begin{matrix} f[i][j-2][0] \\ \vdots \end{matrix} \right\}$$

$$f[i][j][1] = f[i][j-1][0] + f[i][j-2][0] + \dots + f[i][j-(\text{limit}-1)][0] + f[i][j-\text{limit}][0] \quad \rightarrow \text{Eqn(1)}$$

$$f[i][j+1][1] = f[i][j][0] + f[i][j-1][0] + \dots + f[i][j+1-(\text{len}-1)][0] + f[i][j+1-\text{limit}][0] \quad \rightarrow \text{Eqn(2)}$$

$$\text{Eqn(2)} - \text{Eqn(1)}$$

$$f[i][j+1][1] - f[i][j][1] = f[i][j][0] - f[i][j-\text{limit}][0];$$

$$f[i][j+1][1] = f[i][j][1] + f[i][j][0] - f[i][j-\text{limit}][0];$$

↓
Simplification

$$(j+1) \rightarrow j$$

$$f[i][j][1] = f[i][j-1][1] + f[i][j-1][0] - f[i][j-1-\text{limit}][0];$$



$$f[i][j][0] = f[i-1][j][0] + f[i-1][j][1] - f[i-1-limit][j][0]$$



$i = \# \text{ } 0\text{'s}$

limit = 3

$j = \# \text{ } 1\text{'s}$

$$f[i][0][0] = 1$$

$i = \# \text{ } 0\text{'s}$

$j = \# \text{ } 1\text{'s}$

limit

$$f[0][j][1] = 1$$

```
for (i = 0; i <= min(zero, limit); i++) {
```

```
    t[i][0][0] = 1
```

```
}
```

```
for (j = 0; j <= min(one, limit); j++) {
```

```
    t[0][j][i] = 1;
```

```
}
```

```
for (
```

```
)
```

```
    for (
```

```
)
```